

Banco de Problemas - Fase 4 - Componente práctico -**Prácticas Simuladas**

Cristian Fernando Solano Villamizar

Jhonatan Flórez Torres

Mayo 2026

Universidad Nacional abierta y a distancia-UNAD

Fundamentos de Programación

Introducción

El presente trabajo tiene como finalidad desarrollar la actividad correspondiente a la Fase 4 – Componente práctico – Prácticas simuladas del curso de Fundamentos de Programación. La actividad se enfocó en el análisis, depuración y corrección de programas en Python que contenían errores intencionales de sintaxis, lógica y ejecución.

Para el desarrollo del ejercicio seleccionado, se utilizó el apoyo de herramientas de Inteligencia Artificial integradas en Visual Studio Code, específicamente GitHub Copilot, con el propósito de identificar errores, comprender su origen y aplicar soluciones adecuadas. Más que obtener una solución automática, el objetivo fue fortalecer las habilidades de razonamiento lógico, depuración de código y comprensión de estructuras fundamentales de programación.

Objetivos

- Identificar los errores presentes en el programa proporcionado mediante pruebas de ejecución y análisis del código.
- Utilizar herramientas de Inteligencia Artificial para apoyar el proceso de depuración y corrección del programa.
- Aplicar estructuras fundamentales de programación en Python, como funciones, ciclos, condicionales y manejo de excepciones.
- Comprender la importancia de la validación de datos y el control de errores dentro de un sistema informático.
- Documentar los prompts utilizados y las soluciones implementadas durante el proceso de corrección del código.

PROBLEMA 1 REALIZADO POR CRISTIAN SOLANO:**Problema 1:**

Cinemas "Cine Full" desea sistematizar la venta de entradas y la gestión de sus asientos en una única sala. La sala tiene 5 filas y 10 asientos por fila (una matriz de 5 X 10).

Se debe implementar el siguiente menú de opciones:

Encabezado – Menú: Cine Full

Opción 1. Venta de Asiento

Opción 2. Recolección/Devolución de Asiento.

Opción 3. Mostrar Estado de la Sala.

Opción 4. Salir.

Mensaje en pantalla para ingresar la selección: ¿Cuál es su opción?

Implementar las siguientes funciones:

- `inicializar_sala()`: Inicializa la sala como una matriz de 5 X 10

donde:

- o 0 representa un asiento disponible.
- o 1 representa un asiento vendido.
- o 2 representa un asiento reservado (por devolver).
- o Debe retornar la matriz de la sala.

- `mostrar_sala(sala)`: Imprime el estado actual de la sala de forma clara, incluyendo las etiquetas de fila y asiento.

- `validar_asiento(sala, fila, asiento)`: Recibe la matriz sala, la fila (1 a 5) y el asiento (1 a 10).
 - o Debe verificar si la fila y el asiento están dentro del rango válido.
 - o Si son válidos, retorna 1. Si no, retorna 0.

- `vender_asiento(sala, fila, asiento)`: Recibe la matriz sala, la fila y el asiento.
 - o Si el asiento es válido y está disponible (0), lo marca como vendido (1) y retorna el precio del asiento según la fila: Fila 1-2: \$8,000, Fila 3-4: \$6,000, Fila 5: \$4,000.
 - o En cualquier otro caso (inválido o ya vendido/reservado), retorna 0.

- `devolver_asiento(sala, fila, asiento)`: Recibe la matriz sala, la fila y el asiento.
 - o Si el asiento es válido y está vendido (1), lo marca como reservado (2) (para indicar que fue devuelto y está en 2 espera de la devolución del dinero) y retorna la penalidad por devolución: 20% del precio original del asiento.

- o Si el asiento es válido y está reservado (2), lo marca como disponible (0) y retorna \$-1\$ (indicando que la devolución se completó).
- o En cualquier otro caso, retorna 0.

Al final del día (al salir del programa), se debe indicar:

1. Ingreso Total Neto por ventas (ventas - devoluciones completadas).
2. Total de Penalidades acumuladas.

Código fuente Problema 1 sin Solucionar:

```
FILAS = 5
ASIENTOS_POR_FILA = 10

def inicializar_sala():
    fila = [0] * ASIENTOS_POR_FILA
    sala = [fila] * FILAS
    return sala

def mostrar_sala(sala):
    print("\n--- Estado de la Sala ---")
    for f in range(FILAS - 1):
        print(f"F{f+1}:", end=" ")
        for estado in sala[f]:
            simbolo = ""
            if estado == 0:
```

```
    simbolo = " D"

    elif estado == 1:

        simbolo = " V"

    elif estado == 2:

        simbolo = " R"

    print(simbolo, end="")

print()

def validar_asiento(sala, fila, asiento):

    if fila >= 1 and fila <= 5 and asiento >= 0 and asiento < 10:

        return 1

    else:

        return 0

def obtener_precio(fila):

    if fila == 1 or 2:

        return "$8000"

    elif fila == 3 or 4:

        return 6000

    elif fila == 5:

        return 4000

    return 0

def vender_asiento(sala, fila, asiento):

    if validar_asiento(sala, fila, asiento) == 0:

        return 0
```

```
indice_fila = fila - 1

indice_asiento = asiento

if sala[indice_fila][indice_asiento] == 0:
    sala[indice_fila][indice_asiento] = 1
    return obtener_precio(fila)
else:
    return 0

def devolver_asiento(sala, fila, asiento):
    if validar_asiento(sala, fila, asiento) == 0:
        return 0

    indice_fila = fila - 1
    indice_asiento = asiento - 1
    precio_base = obtener_precio(fila)

    if sala[indice_fila][indice_asiento] == 1:
        sala[indice_fila][indice_asiento] = 2
        penalidad = precio_base * 0.20
        return penalidad

    elif sala[indice_fila][indice_asiento] == 2:
        sala[indice_fila][indice_asiento] = 0
        return -1
```



```
else:
```

```
    return 0
```

```
def menu_principal():
```

```
    sala_cine = inicializar_sala()
```

```
    ingreso_neto = 0
```

```
    total_penalizaciones = 0
```

```
    opcion = 0
```

```
    while opcion != 4:
```

```
        print("\n=====")
```

```
        print("  Menú: Cine Full      ")
```

```
        print("=====")
```

```
        print("1. Venta de Asiento.")
```

```
        print("2. Recolección/Devolución de Asiento.")
```

```
        print("3. Mostrar Estado de la Sala.")
```

```
        print("4. Salir.")
```

```
    try:
```

```
        opcion = int(input("¿Cuál es su opción? "))
```

```
    except ValueError:
```

```
        print("Opción no válida.")
```

```
        continue
```

```
    if opcion == 1:
```

```
        print("\n--- VENTA DE ENTRADAS ---")
```

```
try:

    f = int(input("Ingrese el número de Fila (1-5): "))
    a = int(input("Ingrese el número de Asiento (1-10): "))

except ValueError:

    print("Fila o Asiento deben ser números.")

    continue

precio_venta = vender_asiento(sala_cine, f, a)

if precio_venta != 0:

    ingreso_netto += precio_venta

    print(f"Venta exitosa. Asiento F{f}-A{a} vendido por  
{precio_venta}.")

else:

    print(f"Error en la venta.")

elif opcion == 2:

    print("\n--- RECOLECCIÓN/DEVOLUCIÓN ---")

    try:

        f = int(input("Ingrese el número de Fila (1-5): "))
        a = int(input("Ingrese el número de Asiento (1-10): "))

    except ValueError:

        print("Fila o Asiento deben ser números.")

        continue

    resultado = devolver_asiento(sala_cine, f, a)
```

```
if resultado > 0:

    total_penalidades += resultado

    print(f"Devolución solicitada para F{f}-A{a}. Penalidad
aplicada: {resultado}.")

elif resultado == -1:

    precio_base = obtener_precio(f)

    print(f"Devolución completada para F{f}-A{a}.")

else:

    print(f"El asiento F{f}-A{a} no se puede procesar.")


elif opcion == 3:

    mostrar_sala(sala_cine)


elif opcion == 4:

    print("\n--- RESUMEN DEL DÍA ---")

    print(f"Ingreso Total Neto: {ingreso_neto}")

    print(f"Total de Penalidades Acumuladas: {total_penalidades}")

    print("¡Gracias por usar el sistema!")


else:

    print("Opción no reconocida.")


if __name__ == "__main__":

    menu_principal()
```

Prompt 1:

Tengo un programa en Python para un cine con una matriz de 5x10 asientos. El programa presenta errores en la inicialización de la matriz y en la visualización de la sala. Ayúdame a identificar el problema y corregirlo explicando el motivo.

Respuesta:

Problemas encontrados:

Inicialización de matriz (línea 5-7): Usar `[fila] * FILAS` crea referencias a la MISMA lista en lugar de copias independientes. Modificar un asiento afecta todas las filas.

Visualización incompleta (línea 9): `range(FILAS - 1)` solo muestra 4 filas en lugar de 5.

Lógica de precio incorrecta (línea 30): `if fila == 1 or 2:` siempre es verdadero. Debe ser `if fila == 1 or fila == 2:`.

Inconsistencia en índices (línea 64): `devolver_asiento()` resta 1 al asiento, pero `vender_asiento()` no. Debe ser consistente.

Cadena mal cerrada (línea 97): Falta comilla de cierre en el `print` del menú.

Error de entrada de asientos: Se ingresa 1-10 pero no siempre se convierte correctamente a índices 0-9.

Prompt 2:

¿Tengo algún error adicional en la sintaxis?

Respuesta:

Encontré errores de sintaxis críticos:

Línea 108-109: Cadena dividida incorrectamente en dos líneas

Línea 131-132: Otro print dividido entre líneas sin continuación

Línea 23: Validación de asiento incorrecta (valida 0-9 pero pide 1-10)

Prompt 3:

Analiza la función obtener_precio() y el cálculo de devoluciones en este programa de Python. El sistema devuelve precios incorrectos y genera errores en operaciones matemáticas.

Respuesta:**Cambios realizados:**

Problema	Solución
Retorna -1 inválido	Ahora retorna <code>0.80 * precio_base</code> (reembolso del 80%)
Ingreso neto incorrecto	Nueva fórmula: <code>ingreso_total - reembolsos_totales</code>
Sin tracking de reembolsos	Nuevo variable: <code>reembolsos_totales</code>

Problema	Solución
Sin detalle en resumen	Muestra: ventas, penalidades, reembolsos, neto
Lógica de devolución confusa	Ahora: si estado es 2, resta de reembolsos; si no, suma a penalidades

Cálculos correctos ahora:

- Venta: suma completa al ingreso
- Devolución (1→2): cobra penalidad 20%
- Reembolso (2→0): devuelve 80% del precio
- Ingreso Neto = Ventas - Reembolsos

Cambios adoptados:

```
FILAS = 5
ASIENTOS_POR_FILA = 10

def inicializar_sala():
    sala = [[0] * ASIENTOS_POR_FILA for _ in range(FILAS)]
    return sala

def mostrar_sala(sala):
    print("\n--- Estado de la Sala ---")
    for f in range(FILAS):
        print(f"F{f+1}:", end=" ")
        for estado in sala[f]:
            simbolo = ""
```

```
        if estado == 0:
            simbolo = " D"

        elif estado == 1:
            simbolo = " V"

        elif estado == 2:
            simbolo = " R"

        print(simbolo, end="")

    print()

def validar_asiento(sala, fila, asiento):

    if fila >= 1 and fila <= 5 and asiento >= 1 and asiento <= 10:

        return 1

    else:

        return 0

def obtener_precio(fila):

    if fila == 1 or fila == 2:

        return 8000

    elif fila == 3 or fila == 4:

        return 6000

    elif fila == 5:

        return 4000

    return 0

def vender_asiento(sala, fila, asiento):

    if validar_asiento(sala, fila, asiento) == 0:

        return 0
```

```
indice_fila = fila - 1

indice_asiento = asiento - 1

if sala[indice_fila][indice_asiento] == 0:
    sala[indice_fila][indice_asiento] = 1
    return obtener_precio(fila)
else:
    return 0

def devolver_asiento(sala, fila, asiento):
    if validar_asiento(sala, fila, asiento) == 0:
        return 0

    indice_fila = fila - 1
    indice_asiento = asiento - 1
    precio_base = obtener_precio(fila)

    if sala[indice_fila][indice_asiento] == 1:
        sala[indice_fila][indice_asiento] = 2
        penalidad = precio_base * 0.20
        return penalidad

    elif sala[indice_fila][indice_asiento] == 2:
        sala[indice_fila][indice_asiento] = 0
        reembolso = precio_base * 0.80
        return reembolso

    else:
```



```
        return 0

def menu_principal():

    sala_cine = inicializar_sala()

    ingreso_total = 0

    penalidades_cobradas = 0

    reembolsos_totales = 0

    opcion = 0

    while opcion != 4:

        print("\n=====")
        print("  Menú: Cine Full")
        print("=====")
        print("1. Venta de Asiento.")
        print("2. Recolección/Devolución de Asiento.")
        print("3. Mostrar Estado de la Sala.")
        print("4. Salir.")

        try:

            opcion = int(input("¿Cuál es su opción? "))

        except ValueError:

            print("Opción no válida.")

            continue

    if opcion == 1:

        print("\n--- VENTA DE ENTRADAS ---")

        try:

            f = int(input("Ingrese el número de Fila (1-5): "))
```

```
a = int(input("Ingrese el número de Asiento (1-10): "))

except ValueError:

    print("Fila o Asiento deben ser números.")

    continue

precio_venta = vender_asiento(sala_cine, f, a)

if precio_venta != 0:

    ingreso_total += precio_venta

    print(f"Venta exitosa. Asiento F{f}-A{a} vendido por  
${precio_venta}.")

else:

    print(f"Error en la venta.")

elif opcion == 2:

    print("\n--- RECOLECCIÓN/DEVOLUCIÓN ---")

    try:

        f = int(input("Ingrese el número de Fila (1-5): "))

        a = int(input("Ingrese el número de Asiento (1-10): "))

    except ValueError:

        print("Fila o Asiento deben ser números.")

        continue

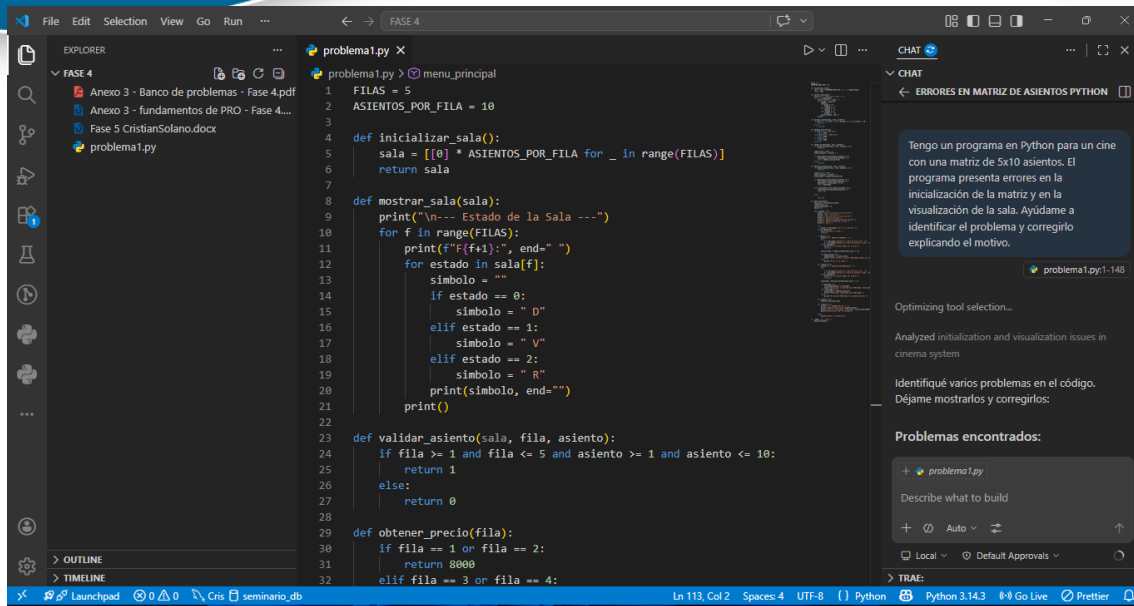
    resultado = devolver_asiento(sala_cine, f, a)

    if resultado > 0:

        if sala_cine[f-1][a-1] == 2:

            penalidades_cobradas += resultado
```

```
        print(f"Devolución solicitada para F{f}-A{a}.\nPenalidad aplicada: ${resultado}.")\n\n        else:\n\n            reembolsos_totales += resultado\n\n            print(f"Devolución completada para F{f}-A{a}.\nReembolso: ${resultado}.")\n\n        else:\n\n            print(f"El asiento F{f}-A{a} no se puede procesar.")\n\n    elif opcion == 3:\n\n        mostrar_sala(sala_cine)\n\n    elif opcion == 4:\n\n        ingreso_netto = ingreso_total - reembolsos_totales\n\n        print("\n--- RESUMEN DEL DÍA ---")\n\n        print(f"Ingresos por Ventas: ${ingreso_total}")\n\n        print(f"Penalidades Cobradas: ${penalidades_cobradas}")\n\n        print(f"Reembolsos Realizados: ${reembolsos_totales}")\n\n        print(f"Ingreso Total Netto: ${ingreso_netto}")\n\n        print("¡Gracias por usar el sistema!")\n\n    else:\n\n        print("Opción no reconocida.")\n\nif __name__ == "__main__":\n\n    menu_principal()
```



```
1 menu_principal
2 FILAS = 5
3 ASIENTOS_POR_FILA = 10
4
5 def inicializar_sala():
6     sala = [[0] * ASIENTOS_POR_FILA for _ in range(FILAS)]
7     return sala
8
9 def mostrar_sala(sala):
10    print("\n-- Estado de la Sala --")
11    for f in range(FILAS):
12        print(f"Fila{f}: ", end=" ")
13        for estado in sala[f]:
14            simbolo = ""
15            if estado == 0:
16                simbolo = "O"
17            elif estado == 1:
18                simbolo = "V"
19            elif estado == 2:
20                simbolo = "R"
21            print(simbolo, end=" ")
22        print()
23
24 def validar_asiento(sala, fila, asiento):
25     if fila >= 1 and fila <= 5 and asiento >= 1 and asiento <= 10:
26         return 1
27     else:
28         return 0
29
30 def obtenerPrecio(fila):
31     if fila == 1 or fila == 2:
32         return 8000
33     elif fila == 3 or fila == 4:
```

¿Qué aprendiste al corregir con IA?

Aprendí a identificar errores de sintaxis, lógica e índices en Python utilizando la IA como apoyo. También comprendí mejor el manejo de matrices, validaciones y estructuras condicionales.

¿Qué límites viste en las sugerencias?

En algunos casos la IA proponía soluciones muy generales y fue necesario revisar manualmente el código para adaptar correctamente las correcciones al problema planteado.

PROBLEMA 2 REALIZADO POR JHONATAN FLOREZ TORRES:

Problema 2: Una pequeña empresa necesita automatizar la gestión y el seguimiento del desempeño de sus ventas para motivar a su equipo. Actualmente, los datos de ventas se manejan de forma manual, lo que dificulta la actualización rápida de los registros y la evaluación oportuna de los incentivos. Se requiere un sistema que pueda llevar un registro de las ventas por mes y determinar si un vendedor califica para un bono basado en un límite de ventas predefinido.

Código:

```
VENTAS_POR_MES = {"enero": 1500, "febrero": 2200, "marzo": 1800}
LIMITE_BONO = 5000

def solicitar_datos():
    """Solicita un nombre y una cantidad al usuario."""
    nombre_vendedor = input("Ingrese su nombre: ")
    try:
        cantidad_nueva = int(input("Ingrese las ventas de
abril: "))
    except:
        print("Entrada inválida, usando 0.")
        cantidad_nueva = 0

    return nombre_vendedor, cantidad_nueva

def agregar_ventas(datos_actuales, mes, monto):
    """Agrega un nuevo
mes de ventas al diccionario."""
    datos_actuales[mes] = monto
    return list(datos_actuales.values())

def revisar_bono(ventas_totales, limite):
```

```
"""Verifica si el vendedor califica para un bono.""" if
ventas_totales > limite:
    monto_bono = ventas_totales /
limite      print(f"¡Felicidades! Gana un bono de:
{monto_bono}") else:
    print("Siga esforzándose para el bono.")

contador = 1 while contador < 3:
    print(f"\n--- Iteración {contador} ---")

    vendedor, nuevas_ventas = solicitar_datos()
    VENTAS_POR_MES_NUEVO = agregar_ventas(VENTAS_POR_MES,
"abril", nuevas_ventas)

    total_anual = sum(VENTAS_POR_MES_NUEVO)
        try:      revisar_bono(total_anual,
LIMITE_BONO_INCORRECTO)      print(f"Ventas de {vendedor}:
{total_anual}. Ventas de mayo:
{VENTAS_POR_MES['mayo']}") except Exception as e:
    print("Ocurrió un problema en el cálculo final.")
```

Prompt 1:

Tengo un código en Python para registrar ventas mensuales de vendedores, pero presenta errores de indentación y sintaxis en los bloques try y while. Ayúdame a identificar los errores y corregirlos explicando qué estaba mal.

Respuesta:

La IA identificó problemas de indentación en el bloque try-except y en el ciclo while.

También encontró que la línea:

```
contador = 1 while contador < 3:
```

tenía un error de sintaxis porque el while debía ir en otra línea.

Se corrigió así:

```
contador = 1
```

```
while contador < 3:
```

Además, se organizaron correctamente los bloques try y except dentro de la función solicitar_datos().

Prompt 2:

Analiza este programa en Python y revisa por qué se produce un error relacionado con la variable LIMITE_BONO_INCORRECTO y con el acceso al diccionario VENTAS_POR_MES['mayo'].

Respuesta:

La IA detectó un error de tipo `NameError` porque la variable:

```
LIMITE_BONO_INCORRECTO
```

no estaba definida.

Se reemplazó por:

```
LIMITE_BONO
```

También indicó que el programa intentaba acceder al mes "mayo" dentro del diccionario, pero ese valor no existía:

```
VENTAS_POR_MES['mayo']
```

Para evitar el error, se eliminó esa impresión y solamente se mostró el total de ventas del vendedor.

Prompt 3:

Revisa la lógica del programa de ventas en Python y verifica si el contador del ciclo se está actualizando correctamente. Además, valida si el cálculo del bono funciona de forma adecuada.

Respuesta:

La IA encontró que el contador nunca aumentaba, lo que podía generar un ciclo infinito.

Se agregó:

```
contador += 1
```


al final del ciclo while.

También verifiqué que el cálculo del bono funcionara correctamente usando:

```
monto_bono = ventas_totales / limite
```

y confirmó que la condición:

```
if ventas_totales > limite:
```

estaba correctamente implementada para determinar si el vendedor ganaba el bono.

CORRECCION

```
VENTAS_POR_MES = {"enero": 1500, "febrero": 2200, "marzo": 1800}
LIMITE_BONO = 5000

def solicitar_datos():
    """Solicita un nombre y una cantidad al usuario."""
    nombre_vendedor = input("Ingrese su nombre: ")
    try:
        cantidad_nueva = int(input("Ingrese las ventas de abril: "))
    except:
        print("Entrada inválida, usando 0.")
        cantidad_nueva = 0

    return nombre_vendedor, cantidad_nueva
```

```
def agregar_ventas(datos_actuales, mes, monto):  
    """Agrega un nuevo mes de ventas al diccionario."""  
    datos_actuales[mes] = monto  
    return list(datos_actuales.values())  
  
def revisar_bono(ventas_totales, limite):  
    """Verifica si el vendedor califica para un bono."""  
    if ventas_totales > limite:  
        monto_bono = ventas_totales / limite  
        print(f"¡Felicidades! Gana un bono de: {monto_bono}")  
    else:  
        print("Siga esforzándose para el bono.")  
  
contador = 1  
  
while contador < 3:  
    print(f"\n--- Iteración {contador} ---")  
  
    vendedor, nuevas_ventas = solicitar_datos()  
    VENTAS_POR_MES_NUEVO = agregar_ventas(VENTAS_POR_MES, "abril",  
nuevas_ventas)  
  
    total_anual = sum(VENTAS_POR_MES_NUEVO)  
  
    try:  
        revisar_bono(total_anual, LIMITE_BONO)  
        print(f"Ventas de {vendedor}: {total_anual}")  
    except Exception as e:
```

```
print("Ocurrió un problema en el cálculo final.")
```

```
contador += 1
```

Conclusiones

- El desarrollo de la actividad permitió fortalecer los conocimientos en programación Python, especialmente en la identificación y corrección de errores de sintaxis, lógica y ejecución presentes en un programa.
- El uso de herramientas de Inteligencia Artificial como GitHub Copilot facilitó el proceso de depuración del código, permitiendo comprender de manera más rápida el origen de algunos errores y posibles soluciones.
- Se evidenció la importancia de validar correctamente los datos ingresados por el usuario y controlar excepciones para evitar fallos durante la ejecución del programa.
- La actividad ayudó a mejorar el razonamiento lógico y la capacidad de análisis frente a problemas de programación, comprendiendo que la IA debe utilizarse como herramienta de apoyo y no como reemplazo del aprendizaje.

Bibliografía

- Downey, A. (2016). Think Python: How to think like a computer scientist (2.ª ed.). O'Reilly Media.
- Gutttag, J. (2021). Introduction to computation and programming using Python (3.ª ed.). MIT Press.
- Python Software Foundation. (2026). Python documentation. Python Documentation
- Microsoft. (2026). Visual Studio Code Documentation. Visual Studio Code
- GitHub Copilot (2026). GitHub Copilot documentation. GitHub.